FIAP ON

COLHEITA DE SOLUÇÕES INTELIGENTES - TRANSFORMANDO DADOS EM AÇÕES

A ARTE DO AGRUPAMENTO COM DRS

4

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 1 - Representação de uma função de grupo 18
Figura 2 - Resultado do SELECT com COUNT(*) 19
Figura 3 - Resultado do SELECT com COUNT(coluna) 19
Figura 4 - Resultado do SELECT com COUNT(*) com validação de nulos 20
Figura 5 - Resultado do SELECT com COUNT(coluna) e cláusula DISTINCT 21
Figura 6 - Resultado do SELECT com SUM(coluna) 22
Figura 7 - Resultado do SELECT com AVG (coluna) 23
Figura 8 - Resultado do SELECT com MIN(coluna) 24
Figura 9 - Resultado do SELECT com MAX(coluna) 25
Figura 10 - Resultado do SELECT com STDDEV(coluna) 26
Figura 11 - Resultado do SELECT com VARIANCE(coluna) 27
Figura 12 - Representação de um agrupamento 28
Figura 13 - Resultado do SELECT com GROUP BY 29
Figura 14 - Erros durante a execução de agrupamentos 29
Figura 15 - Resultado do SELECT com GROUP BY e função de grupo 30
Figura 16 - Resultado do SELECT com GROUP BY e várias tabelas 32
Figura 17 - Resultado do SELECT com o GROUP BY com HAVING 33
Figura 18 - Erros durante a execução de agrupamentos com WHERE 34
Figura 19 - Resultado do SELECT com a instrução CASE 35
Figura 20 - Resultado da Consulta 1 37
Figura 21 - Resultado da Consulta 2 38
Figura 22 - Resultado da Consulta 3 38
Figura 23 - Resultado da Consulta 4 39
Figura 24 - Resultado da Consulta 5 40
Figura 25 - Resultado da Consulta 6 41

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE QUADROS

Quadro 1 - Funções de grupo 7

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Exemplo de código-fonte SQL – Comando SELECT com agrupamento 7
Código-fonte 2 - SCRIPT DE CRIAÇÃO DO BANCO DE DADOS....11
Código-fonte 3 - Script para popular as tabelas do banco de dados SIP 18
Código-fonte 4 - Exemplo de código-fonte SQL – Função COUNT(*)....19
Código-fonte 5 - Exemplo de código-fonte SQL – Função COUNT(coluna) 19
Código-fonte 6 - Exemplo de código-fonte SQL – Função COUNT(*) com validação de nulos....20
Código-fonte 7 - Exemplo de código-fonte SQL – Função COUNT(coluna) e cláusula DISTINCT....20
Código-fonte 8 - Exemplo de código-fonte SQL – Função SUM(coluna) 21
Código-fonte 9 - Exemplo de código-fonte SQL – Função AVG (coluna)....22
Código-fonte 10 - Exemplo de código-fonte SQL – Função MIN(coluna)....23
Código-fonte 11 - Exemplo de código-fonte SQL – Função MAX(coluna) 24
Código-fonte 12 - Exemplo de código-fonte SQL – Função STDDEV(coluna)....25
Código-fonte 13 - Exemplo de código-fonte SQL – Função VARIANCE(coluna)....26
Código-fonte 14 - Exemplo de código-fonte SQL – GROUP BY 28
Código-fonte 15 - Exemplo de código-fonte SQL – GROUP BY e função de grupo .30
Código-fonte 16 - Exemplo de código-fonte SQL – GROUP BY e várias tabelas 31
Código-fonte 17 - Exemplo de código-fonte SQL – GROUP BY com HAVING....33
Código-fonte 18 - Exemplo de instrução CASE....34
Código-fonte 19 - Exemplo de código-fonte SQL – Instrução CASE....35
Código-fonte 20 - Exemplo de código-fonte SQL – comando SELECT com função de grupo – Consulta 1 36
Código-fonte 21 - Exemplo de código-fonte SQL – comando SELECT com função de grupo e agrupamento – Consulta 2 37
Código-fonte 22 - Exemplo de código-fonte SQL – comando SELECT com função de grupo e agrupamento – Consulta 3 38
Código-fonte 23 - Exemplo de código-fonte SQL – comando SELECT com função de grupo e agrupamento – Consulta 4 39
Código-fonte 24 - Exemplo de código-fonte SQL – comando SELECT com função de grupo e agrupamento – Consulta 5 40
Código-fonte 25 - Exemplo de código-fonte SQL – comando SELECT com função de grupo e agrupamento – Consulta 6 41

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

1 A ARTE DO AGRUPAMENTO COM DRS 6
1.1 Funções de Grupo e Agrupamentos 6
1.2 Comando SELECT, sintaxe básica para manipulação de grupos de dados 7
1.3 Exemplos de aplicação do comando SELECT utilizando Grupos e Agrupamentos 8
1.4 Visualizando a aplicação de uma função de grupo 18
1.5 Função de grupo: COUNT() 18
1.6 Função de grupo: SUM() 21
1.7 Função de grupo: AVG() 22
1.8 Função de grupo: MIN() 23
1.9 Função de grupo: MAX() 24
1.10 Função de grupo: STDDEV() 25
1.11 Função de grupo: VARIANCE() 26
1.12 Visualizando a aplicação de um agrupamento 27
1.13 Criando um agrupamento de dados 28
1.14 Criando um agrupamento de dados apenas com funções de grupo 29
1.15 Criando um agrupamento de dados com várias tabelas 31
1.16 Cláusula HAVING 32
1.17 Desvio Condicional (Instrução CASE) 34
1.18 Hands On – Consulta ou Recuperação de dados utilizando Grupos e Agrupamentos 36
1.18.1 Estudo de Caso: SIP – SISTEMA DE CONTROLE DE IMPLANTACAO DE PROJETOS 36
1.18.1.1 Recuperando dados utilizando Grupos e Agrupamentos 36
REFERÊNCIAS 42
GLOSSÁRIO 43

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1 A ARTE DO AGRUPAMENTO COM DRS

Neste capítulo, avançaremos um pouco mais nos comandos DRS e trataremos de grupo e agrupamento, conheça as diversas funções de grupo neste capítulo!

1.1 Funções de Grupo e Agrupamentos

Na linguagem SQL, temos instruções que nos permitem recuperar os dados de tabelas de um banco de dados. Essas instruções pertencem à divisão DRS (Data Retrieve Statement) ou DQL (Data Query Language) da linguagem SQL.

SELECT, a partir de agrupamentos (GROUP BY), nos permite a pesquisa através de agrupamento de registros. Um agrupamento representa a união de um ou mais registros em um único resultado, de tal forma que os registros sejam agrupados.

As funções de grupo operam em um conjunto de linhas/registros, visando fornecer um resultado para o grupo criado. As operações das funções de grupo podem envolver todas as linhas de uma tabela ou conjuntos de linhas definidos por critérios preestabelecidos.

É necessário o uso de funções de grupo, por exemplo: MIN, MAX, AVG, COUNT, SUM, como uma coluna a ser apresentada pelo comando SELECT.

Como em outras linguagens, funções requerem argumentos para realizar operações e retornar valores. Na linguagem SQL, as funções de grupo esperam como argumento o nome de uma coluna: FUNÇÃO (coluna).

As funções de grupo são:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

FUNÇÃO DE GRUPO	DESCRIÇÃO
COUNT()	Retorna número de linhas afetadas pelo comando.
SUM()	Retorna a somatória do valor das colunas especificadas.
AVG()	Retorna a média aritimética dos valores das colunas.
MIN()	Retorna o menor valor da coluna de um grupo de linhas.
MAX()	Retorna o maior valor da coluna de um grupo de linhas.
STDDEV()	Retorna o desvio padrão da coluna.
VARIANCE()	Retorna a variância da coluna.

Quadro 1 - Funções de grupo
Fonte: Elaborado pela autora (2017)

1.2 Comando SELECT, sintaxe básica para manipulação de grupos de dados

Para recuperarmos os dados das tabelas de um banco de dados, utilizamos o comando SELECT, que é um comando dentro da divisão DRS/DQL, da linguagem SQL. Para manipularmos grupos de dados, devemos inserir cláusulas adicionais à estrutura do comando SELECT.

O agrupamento simples envolve todo o conjunto de uma determinada tabela, isto é, considera todas as linhas que satisfazem a uma condição e cada função envolvida produz um único resultado para o conjunto.

Sintaxe básica do comando SELECT para manipulação de grupos de dados:

```
      SELECT [ coluna, ] funçao_de_grupo (coluna)
        FROM tabela
     [ WHERE condição ]
  [ GROUP BY coluna ]
    [ HAVING condiçao ]
[   ORDER BY coluna [, coluna, ...]
```

Código-fonte 1 - Exemplo de código-fonte SQL – Comando SELECT com agrupamento
Fonte: Puga, França e Goya (2013)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Onde:

- ***SELECT*:** especifica as colunas (campos) necessárias para a pesquisa.
- ***FUNÇÃO DE GRUPO [coluna]*:** indica o nome da função que será utilizada e cujos dados da coluna definida serão consideradas como parâmetro para a função.
- ***FROM*:** indica em que tabelas serão efetuadas estas pesquisas.
- ***WHERE*:** condição para que se execute a pesquisa (filtra dados).
- ***ORDER BY*:** especifica em que ordem deverá ser apresentada a pesquisa desejada, por qual campo estará ordenada (de forma crescente ou decresencente).
- ***GROUP By [coluna]*:** permite criar os grupos de dados.
- ***HAVING [condição]*:** similar à cláusula WHERE, aplicado somente a agrupamentos. Permite limitar (filtrar) os grupos a serem recuperados.
- ***<nome-tabela>:*** tabela que terá os dados recuperados.
- ***<nome-coluna>:*** colunas que serão recuperadas da tabela.
- ***<condição>:*** condição (filtro) utilizado no momento da recuperação dos dados da tabela.

Nota: a cláusula **GROUP BY** deve vir antes do **ORDER BY** e após o **WHERE**. A lista de colunas que se quer agrupar deve corresponder à mesma sequência da cláusula **GROUP BY.**

1.3 Exemplos de aplicação do comando SELECT utilizando Grupos e Agrupamentos

Para exemplificar as aplicações dos comandos DRS/DQL, iremos construir (implementar) e popular uma estrutura de banco de dados.

Como sugestão, podemos utilizar a ferramenta SQL Developer da Oracle.

O nome do SCRIPT (arquivo com comandos SQL) deve possuir um nome que ajude na sua identificação, por exemplo: “SCRIPT_DDL_PROJETO_SIP.SQL”, com a extensão SQL.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Apresentamos o código na linguagem SQL, que permite implementar a estrutura de banco de dados do PROJETO SIP.

```
-- SCRIPT PARA CRIAÇÃO DO BANCO DE DADOS

-- DO ESTUDO DE CASO SIP

-- COMANDOS PARA EXCLUIR AS TABELAS

DROP TABLE t_sip_departamento CASCADE CONSTRAINTS;
DROP TABLE t_sip_dependente CASCADE CONSTRAINTS;
DROP TABLE t_sip_funcionario CASCADE CONSTRAINTS;
DROP TABLE t_sip_implantacao CASCADE CONSTRAINTS;
DROP TABLE t_sip_projeto CASCADE CONSTRAINTS;

-- COMANDOS PARA CRIAR AS TABELAS

-- E ADICIONAR CONSTRAINTS: PRIMARY KEY, CHECK E UNIQUE

CREATE TABLE t_sip_departamento (
    cd_depto NUMBER(3)       NOT NULL,
    nm_depto VARCHAR2(30)    NOT NULL,
    sg_depto CHAR(3)         NOT NULL
);

COMMENT ON COLUMN t_sip_departamento.cd_depto IS
    'Um departamento é identificado por um código numérico de
três dígitos.';

   ALTER TABLE t_sip_departamento
ADD CONSTRAINT pk_sip_departamento
   PRIMARY KEY ( cd_depto );

   ALTER TABLE t_sip_departamento
ADD CONSTRAINT un_sip_depto_nome
        UNIQUE ( nm_depto );

   ALTER TABLE t_sip_departamento
ADD CONSTRAINT un_sip_depto_sigla
        UNIQUE ( sg_depto );

CREATE TABLE t_sip_dependente (
    nr_matricula  NUMBER(5)      NOT NULL,
    cd_dependente NUMBER(2)      NOT NULL,
    nm_dependente VARCHAR2(50)   NOT NULL,
    dt_nascimento DATE           NOT NULL
);
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
   ALTER TABLE t_sip_dependente
ADD CONSTRAINT pk_sip_dependente
   PRIMARY KEY ( nr_matricula, cd_dependente );

CREATE TABLE t_sip_funcionario (
    nr_matricula      NUMBER(5)      NOT NULL,
    cd_depto          NUMBER(3)      NOT NULL,
    nm_funcionario    VARCHAR2(50)   NOT NULL,
    dt_nascimento     DATE           NOT NULL,
    dt_admissao       DATE           NOT NULL,
    ds_endereco       VARCHAR2(100)  NOT NULL,
    vl_salario_mensal NUMBER(7, 2) NOT NULL
);

   ALTER TABLE t_sip_funcionario
ADD CONSTRAINT ck_sip_func_salario
         CHECK ( vl_salario_mensal >= 1212 );

   ALTER TABLE t_sip_funcionario
ADD CONSTRAINT pk_sip_funcionario
   PRIMARY KEY ( nr_matricula );

CREATE TABLE t_sip_implantacao (
    cd_projeto     NUMBER(5) NOT NULL,
    cd_implantacao NUMBER(3) NOT NULL,
    nr_matricula   NUMBER(5) NOT NULL,
    dt_entrada     DATE      NOT NULL,
    dt_saida       DATE          NULL
);

   ALTER TABLE t_sip_implantacao
ADD CONSTRAINT ck_sip_implant_data
         CHECK ( dt_saida > dt_entrada );

   ALTER TABLE t_sip_implantacao
ADD CONSTRAINT pk_sip_implantacao
   PRIMARY KEY ( cd_projeto, cd_implantacao );

CREATE TABLE t_sip_projeto (
    cd_projeto NUMBER(5)    NOT NULL,
    nm_projeto VARCHAR2(40) NOT NULL,
    dt_inicio  DATE         NOT NULL,
    dt_termino DATE
);

-- ADICIONAR CONSTRAINT FOREIGN KEY

   ALTER TABLE t_sip_projeto
ADD CONSTRAINT ck_sip_projeto_data
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
          CHECK ( dt_termino > dt_inicio );

   ALTER TABLE t_sip_projeto
ADD CONSTRAINT pk_sip_projeto
   PRIMARY KEY ( cd_projeto );

   ALTER TABLE t_sip_projeto
ADD CONSTRAINT un_sip_projeto_nome
        UNIQUE ( nm_projeto );

   ALTER TABLE t_sip_dependente
ADD CONSTRAINT fk_sip_dependente_func
   FOREIGN KEY ( nr_matricula )
    REFERENCES t_sip_funcionario ( nr_matricula );

   ALTER TABLE t_sip_funcionario
ADD CONSTRAINT fk_sip_func_depto
   FOREIGN KEY ( cd_depto )
    REFERENCES t_sip_departamento ( cd_depto );

   ALTER TABLE t_sip_implantacao
ADD CONSTRAINT fk_sip_implant_func
   FOREIGN KEY ( nr_matricula )
    REFERENCES t_sip_funcionario ( nr_matricula );

   ALTER TABLE t_sip_implantacao
ADD CONSTRAINT fk_sip_implant_projeto
   FOREIGN KEY ( cd_projeto )
    REFERENCES t_sip_projeto ( cd_projeto );
```

Código-fonte 2 - SCRIPT DE CRIAÇÃO DO BANCO DE DADOS
Fonte: Elaborado pela autora (2023)

Agora iremos popular toda a estrutura do banco de dados referente ao projeto SIP.

O nome do SCRIPT (arquivo com comandos SQL) deve possuir um nome que ajude na sua identificação, por exemplo: “SCRIPT_DML_PROJETO_SIP.SQL”, com a extensão SQL.

Apresentamos o código na linguagem SQL, que permite popular a estrutura de banco de dados do PROJETO SIP:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
-- SCRIPT PARA POPULAR O BD IMPLANTACAO DE PROJETOS

-- ELIMINA O CONTEUDO DAS TABELAS
/*
DELETE FROM T_SIP_IMPLANTACAO;
DELETE FROM T_SIP_PROJETO;
DELETE FROM T_SIP_DEPENDENTE;
DELETE FROM T_SIP_FUNCIONARIO;
DELETE FROM T_SIP_DEPARTAMENTO;
*/

-- POPULAR A TABELA DEPTO COM 5 DEPARTAMENTOS
INSERT INTO T_SIP_DEPARTAMENTO
     VALUES (1, 'FINANCEIRO', 'FIN');
INSERT INTO T_SIP_DEPARTAMENTO
     VALUES (2, 'TECNOLOGIA DA INFORMAÇÃO','TIN');
INSERT INTO T_SIP_DEPARTAMENTO
     VALUES (3, 'CONTAS A PAGAR','CPG');
INSERT INTO T_SIP_DEPARTAMENTO
     VALUES (4, 'FATURAMENTO','FAT');
INSERT INTO T_SIP_DEPARTAMENTO
     VALUES (5, 'RECURSOS HUMANOS', 'RHU');

-- POPULAR A TABELA FUNCIONARIO COM 3 FUNCIONARIOS PARA CADA
DEPTO
-- DEPTO = 1
INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12345,         1,         'JOAO         DA         SILVA',
TO_DATE('10/05/1985','DD/MM/YYYY'),
     TO_DATE('15/09/2012','DD/MM/YYYY'),'RUA X, 49', 5684.66);

INSERT INTO T_SIP_FUNCIONARIO VALUES
```

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
    (12346,        1,        'MANUEL        DA        SILVA',
TO_DATE('05/10/1998','DD/MM/YYYY'),
     TO_DATE('10/11/2015','DD/MM/YYYY'),'RUA X, 31', 3542.11);

INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12347,        1,        'JANDIRA        DA        SILVA',
TO_DATE('10/12/2000','DD/MM/YYYY'),
     TO_DATE('15/09/2018','DD/MM/YYYY'),'RUA X, 25', 1875.96);

-- DEPTO = 2
INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12348,        2,        'KATIA        REGINA        SOUZA',
TO_DATE('15/01/1995','DD/MM/YYYY'),
     TO_DATE('03/10/2015','DD/MM/YYYY'),'RUA Y, 49', 3894.63);

INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12349,      2,      'MARIA      DAS      DORES      SOUZA',
TO_DATE('18/08/1983','DD/MM/YYYY'),
    TO_DATE('23/10/2017','DD/MM/YYYY'),'RUA Y, 35', 1542.55);

INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12350,        2,        'ALFREDO        DE        SOUZA',
TO_DATE('04/05/1999','DD/MM/YYYY'),
    TO_DATE('05/06/2014','DD/MM/YYYY'),'RUA Y, 27', 5874.52);

-- DEPTO = 3
INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12351,        3,        'GISELE        DE        JESUS',
TO_DATE('15/04/1999','DD/MM/YYYY'),
    TO_DATE('20/03/2017','DD/MM/YYYY'),'RUA Z, 49', 1360.66);

INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12352,        3,        'RAFAEL        DE        JESUS',
TO_DATE('10/08/1998','DD/MM/YYYY'),
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
    TO_DATE('10/08/2012','DD/MM/YYYY'),'RUA Z, 55', 2563.44);

INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12353,        3,        'ROSANA        DE        JESUS',
TO_DATE('14/03/1987','DD/MM/YYYY'),
    TO_DATE('15/08/2019','DD/MM/YYYY'),'RUA Z, 79', 4879.55);

-- DEPTO = 4
INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12354,       4,       'JOSEFINA       DE       ALMEIDA',
TO_DATE('16/10/1997','DD/MM/YYYY'),
    TO_DATE('25/03/2013','DD/MM/YYYY'),'RUA Y, 33', 4561.88);

INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12355,       4,       'LUCIANA       DE       ALMEIDA',
TO_DATE('10/02/1984','DD/MM/YYYY'),
    TO_DATE('28/09/2011','DD/MM/YYYY'),'RUA Y, 44', 2345.52);

INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12356,       4,       'THIAGO       DE       ALMEIDA',
TO_DATE('10/03/1998','DD/MM/YYYY'),
    TO_DATE('24/10/2018','DD/MM/YYYY'),'RUA Y, 55', 1254.22);

-- DEPTO = 5
INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12357,       5,       'LARISSSA       DE       CAMARGO',
TO_DATE('14/02/1997','DD/MM/YYYY'),
    TO_DATE('04/08/2015','DD/MM/YYYY'),'RUA V, 22', 1245.55);

INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12358,       5,       'ANTONIO       DE       CAMARGO',
TO_DATE('25/01/1985','DD/MM/YYYY'),
    TO_DATE('12/08/2016','DD/MM/YYYY'),'RUA V, 44', 2451.33);
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12359,        5,        'JOSE        DE        CAMARGO',
TO_DATE('23/10/1998','DD/MM/YYYY'),
    TO_DATE('20/04/2017','DD/MM/YYYY'),'RUA V, 88', 6541.22);

-- POPULAR A TABELA DEPENDENTE. ESCOLHER 3 FUNCIONARIOS E
CADASTRAR 3 DEPENDENTES
-- PARA CADA FUNCIONARIO ESCOLHIDO
-- FUNCIONARIO = 12345
INSERT INTO T_SIP_DEPENDENTE VALUES
(12345, 1, 'JOANINHA', TO_DATE('12/10/2010','DD/MM/YYYY'));
INSERT INTO T_SIP_DEPENDENTE VALUES
(12345, 2, 'JULINHA', TO_DATE('15/10/2012','DD/MM/YYYY'));
INSERT INTO T_SIP_DEPENDENTE VALUES
(12345, 3, 'TONINHO', TO_DATE('22/10/2014','DD/MM/YYYY'));

-- FUNCIONARIO = 12356
INSERT INTO T_SIP_DEPENDENTE VALUES
(12356, 1,'JUNINHO', TO_DATE('02/10/2011','DD/MM/YYYY'));
INSERT INTO T_SIP_DEPENDENTE VALUES
(12356, 2, 'ZEZINHO', TO_DATE('15/10/2013','DD/MM/YYYY'));
INSERT INTO T_SIP_DEPENDENTE VALUES
(12356, 3, 'MARCELINHO', TO_DATE('17/10/2015','DD/MM/YYYY'));

-- FUNCIONARIO = 12359
INSERT INTO T_SIP_DEPENDENTE VALUES
(12359, 1,'MARIAZINHA', TO_DATE('11/06/2014','DD/MM/YYYY'));
INSERT INTO T_SIP_DEPENDENTE VALUES
(12359, 2,'LUIZINHA', TO_DATE('21/06/2016','DD/MM/YYYY'));
INSERT INTO T_SIP_DEPENDENTE VALUES
(12359, 3,'CARMINHA', TO_DATE('18/06/2018','DD/MM/YYYY'));

-- POPULAR A TABELA PROJETO COM 5 PROJETOS
-- 3 PROJETOS SEM A DATA TÉRMINO
```

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
INSERT INTO T_SIP_PROJETO VALUES
(10001,   'PROJETO   1',   TO_DATE('11/10/2016','DD/MM/YYYY'),
NULL);
INSERT INTO T_SIP_PROJETO VALUES
(10002,   'PROJETO   2',   TO_DATE('25/10/2013','DD/MM/YYYY'),
TO_DATE('15/04/2015','DD/MM/YYYY'));
INSERT INTO T_SIP_PROJETO VALUES
(10003,   'PROJETO   3',   TO_DATE('28/04/2012','DD/MM/YYYY'),
NULL);
INSERT INTO T_SIP_PROJETO VALUES
(10004,   'PROJETO   4',   TO_DATE('25/03/2017','DD/MM/YYYY'),
NULL);
INSERT INTO T_SIP_PROJETO VALUES
(10005,   'PROJETO   5',   TO_DATE('28/09/2014','DD/MM/YYYY'),
TO_DATE('15/09/2018','DD/MM/YYYY'));

-- POPULAR A TABELA IMPLANTACAO
-- PROJETO 1
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10001,    1,    12345,    TO_DATE('11/10/2016','DD/MM/YYYY'),
TO_DATE('15/09/2017','DD/MM/YYYY'));
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10001, 2, 12346, TO_DATE('11/10/2016','DD/MM/YYYY'), NULL);
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10001,    3,    12348,    TO_DATE('11/10/2016','DD/MM/YYYY'),
TO_DATE('25/06/2018','DD/MM/YYYY'));
-- PROJETO 2
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10002,    1,    12345,    TO_DATE('25/10/2013','DD/MM/YYYY'),
TO_DATE('15/04/2014','DD/MM/YYYY'));
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10002,    2,    12352,    TO_DATE('25/10/2013','DD/MM/YYYY'),
TO_DATE('25/07/2014','DD/MM/YYYY'));
INSERT INTO T_SIP_IMPLANTACAO VALUES
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
(10002,    3,    12355,    TO_DATE('25/10/2013','DD/MM/YYYY'),
TO_DATE('15/04/2015','DD/MM/YYYY'));
-- PROJETO 3
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10003, 1, 12354, TO_DATE('28/04/2013','DD/MM/YYYY'), NULL);
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10003, 2, 12355, TO_DATE('28/04/2013','DD/MM/YYYY'), NULL);
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10003, 3, 12345, TO_DATE('28/04/2013','DD/MM/YYYY'), NULL);
-- PROJETO 4
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10004, 1, 12357, TO_DATE('25/03/2017','DD/MM/YYYY'), NULL);
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10004, 2, 12358, TO_DATE('25/03/2017','DD/MM/YYYY'), NULL);
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10004,    3,    12354,    TO_DATE('25/03/2017','DD/MM/YYYY'),
TO_DATE('15/08/2018','DD/MM/YYYY'));
-- PROJETO 5
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10005,    1,    12345,    TO_DATE('28/09/2014','DD/MM/YYYY'),
TO_DATE('15/09/2018','DD/MM/YYYY'));
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10005,    2,    12350,    TO_DATE('28/09/2014','DD/MM/YYYY'),
TO_DATE('15/09/2018','DD/MM/YYYY'));
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10005,    3,    12354,    TO_DATE('28/09/2014','DD/MM/YYYY'),
TO_DATE('15/09/2018','DD/MM/YYYY'));

-- CONSULTAR TABELAS
SELECT * FROM T_SIP_DEPARTAMENTO;
SELECT * FROM T_SIP_FUNCIONARIO;
SELECT * FROM T_SIP_DEPENDENTE;
SELECT * FROM T_SIP_PROJETO;
SELECT * FROM T_SIP_IMPLANTACAO;
```

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
--EFETIVAR AS TRANSAÇÕES PENDENTES
COMMIT;
```

Código-fonte 3 - Script para popular as tabelas do banco de dados SIP
Fonte: Elaborado pela autora (2023)

1.4 Visualizando a aplicação de uma função de grupo

Para que seja possível contar a quantidade de registros de uma determinada coluna, por exemplo, precisamos aplicar a função de grupo COUNT(), informando o argumento. No exemplo, o argumento seria a coluna “CD_DEPTO”.

A imagem a seguir ilustra a contagem de linhas/registros, feita através da função e o retorno, ou seja, a contagem retornada através da função de grupo.

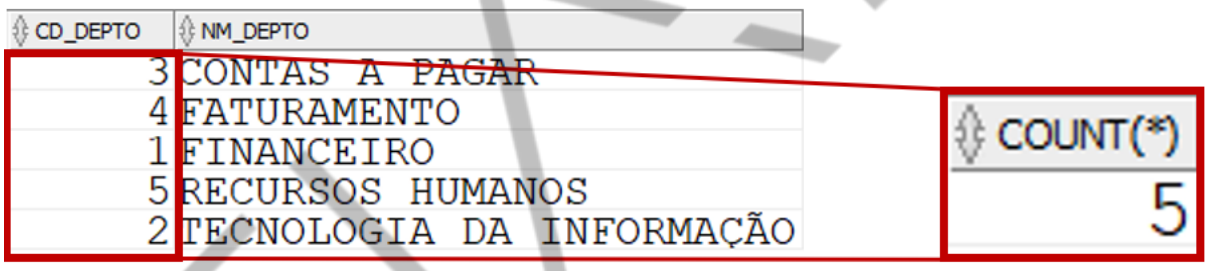


Figura 1 - Representação de uma função de grupo
Fonte: Elaborado pela autora (2023)

1.5 Função de grupo: COUNT()

A função de grupo COUNT() retorna número de linhas afetadas através do comando SELECT.

A função COUNT () pode ser utilizada de duas maneiras:

- **COUNT (*):** retorna o número de linhas/registros de uma tabela, inclusive linhas duplicadas e linhas contendo nulos em alguma coluna.
- **COUNT (coluna):** retorna o número de linhas não nulas da coluna, identificada por coluna.

O comando SELECT, através da função COUNT(*), retornará a quantidade total de linhas/registros da tabela “DEPARTAMENTO”.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
-- EXEMPLO – FUNÇÃO COUNT (*)
SELECT COUNT(*) FROM T_SIP_DEPARTAMENTO;
```

Código-fonte 4 - Exemplo de código-fonte SQL – Função COUNT(*)
Fonte: Elaborado pela autora (2017)

Resultado da execução do comando “SELECT” com a função COUNT(*).

CD_DEPTO	NM_DEPTO
3	CONTAS A PAGAR
4	FATURAMENTO
1	FINANCEIRO
5	RECURSOS HUMANOS
2	TECNOLOGIA DA INFORMAÇÃO

COUNT(*)
5

Figura 2 - Resultado do SELECT com COUNT(*)
Fonte: Elaborado pela autora (2023)

O comando SELECT, através da função COUNT(coluna), retornará a quantidade total de linhas/registros da coluna especificada, com o conteúdo diferente de nulo.

```
-- EXEMPLO – FUNÇÃO COUNT (coluna)
SELECT COUNT(DT_TERMINO) FROM T_SIP_PROJETO;
```

Código-fonte 5 - Exemplo de código-fonte SQL – Função COUNT(coluna)
Fonte: Elaborado pela autora (2017)

Resultado da execução do comando “SELECT” com a função COUNT(coluna).

CD_PROJETO	NM_PROJETO	DT_INICIO	DT_TERMINO
10006	PROJETO 6	28/11/21	(null)
10001	PROJETO 1	11/10/16	(null)
10002	PROJETO 2	25/10/13	15/04/15
10003	PROJETO 3	28/04/12	(null)
10004	PROJETO 4	25/03/17	(null)
10005	PROJETO 5	28/09/14	15/09/18

COUNT(DT_TERMINO)
2

Figura 3 - Resultado do SELECT com COUNT(coluna)
Fonte: Elaborado pela autora (2023)

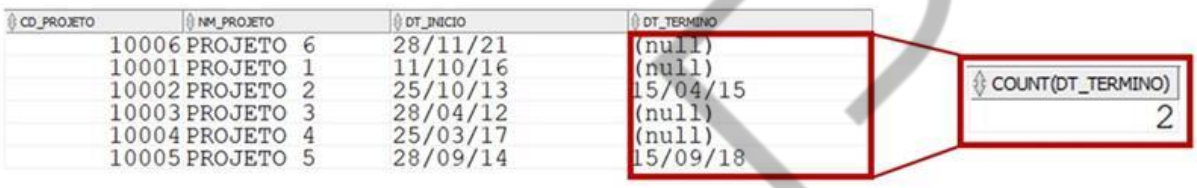
O comando SELECT, através da função COUNT(*), retornará a quantidade total de linhas/registros da tabela “DEPARTAMENTO”. Acrescentando a validação IS NULL ou IS NOT NULL teremos um comando equivalente ao exemplo anterior, no qual informamos a coluna, ou seja, não serão considerados os valores nulos, na recuperação da execução da função de grupo.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
-- EXEMPLO – FUNÇÃO COUNT (*) e validação de NULO
SELECT COUNT(CD_PROJETO)
  FROM T_SIP_PROJETO
 WHERE DT_TERMINO IS NOT NULL;
```

Código-fonte 6 - Exemplo de código-fonte SQL – Função COUNT(*) com validação de nulos
Fonte: Elaborado pela autora (2023)

Resultado da execução do comando “SELECT” com a função COUNT(coluna).



CD_PROJETO	NM_PROJETO	DT_INICIO	DT_TERMINO
10006	PROJETO 6	28/11/21	(null)
10001	PROJETO 1	11/10/16	(null)
10002	PROJETO 2	25/10/13	15/04/15
10003	PROJETO 3	28/04/12	(null)
10004	PROJETO 4	25/03/17	(null)
10005	PROJETO 5	28/09/14	15/09/18

COUNT(DT_TERMINO)
2

Figura 4 - Resultado do SELECT com COUNT(*) com validação de nulos
Fonte: Elaborado pela autora (2023)

O comando SELECT, através da função COUNT(coluna), retornará a quantidade total de linhas/registros da coluna especificada, com o conteúdo diferente de nulo. A cláusula DISTINCT, inserida no comando, ainda recuperará os valores da coluna de forma distinta, ou seja, sem repetição.

```
-- EXEMPLO – FUNÇÃO COUNT (coluna) e cláusula DISTINCT
SELECT COUNT(DISTINCT CD_PROJETO)
  FROM T_SIP_IMPLANTACAO ;
```

Código-fonte 7 - Exemplo de código-fonte SQL – Função COUNT(coluna) e cláusula DISTINCT
Fonte: Elaborado pela autora (2017)

Resultado da execução do comando “SELECT” com a função COUNT(coluna) e a cláusula DISTINCT.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

CD_PROJETO	CD_IMPLANTACAO	NR_MATRICULA	DT_ENTRADA	DT_SAIDA
10001	1	12345	11/10/16	15/09/17
10001	2	12346	11/10/16	(null)
10001	3	12348	11/10/16	25/06/18
10002	1	12345	25/10/13	15/04/14
10002	2	12352	25/10/13	25/07/14
10002	3	12355	25/10/13	15/04/15
10003	1	12354	28/04/13	(null)
10003	2	12355	28/04/13	(null)
10003	3	12345	28/04/13	(null)
10004	1	12357	25/03/17	(null)
10004	2	12358	25/03/17	(null)
10004	3	12354	25/03/17	15/08/18
10005	1	12345	28/09/14	15/09/18
10005	2	12350	28/09/14	15/09/18
10005	3	12354	28/09/14	15/09/18

COUNT(DISTINCTCD_PROJETO)
5

Figura 5 - Resultado do SELECT com COUNT(coluna) e cláusula DISTINCT
Fonte: Elaborado pela autora (2023)

1.6 Função de grupo: SUM()

A função de grupo SUM() retorna o somatório dos valores da coluna especificada através de um comando SELECT. Este comando, através da função SUM(coluna), retornará o somatório dos valores das linhas/registros da coluna especificada, com o conteúdo diferente de nulo.

No exemplo a seguir, será recuperado o somatório de salários de todos os funcionários.

```
-- EXEMPLO – FUNÇÃO SUM (coluna)
SELECT SUM(VL_SALARIO_MENSAL) FROM T_SIP_FUNCIONARIO;
```

Código-fonte 8 - Exemplo de código-fonte SQL – Função SUM(coluna)
Fonte: Elaborado pela autora (2023)

Resultado da execução do comando “SELECT” com a função SUM(coluna).

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUM(VL_SALARIO_MENSAL)
49617,8

NR_MATRICULA	CD_DEPTO	NM_FUNCIONARIO	DT_NASCIMENTO	DT_ADMISSAO	DS_ENDERECO	VL_SALARIO_MENSAL
12345	1	JOAO DA SILVA	10/05/85	15/09/12	RUA X, 49	5684,66
12346	1	MANUEL DA SILVA	05/10/98	10/11/15	RUA X, 31	3542,11
12347	1	JANDIRA DA SILVA	10/12/00	15/09/18	RUA X, 25	1875,96
12348	2	KATIA REGINA SOUZA	15/01/95	03/10/15	RUA Y, 49	3894,63
12349	2	MARIA DAS DORES SOUZA	18/08/83	23/10/17	RUA Y, 35	1542,55
12350	2	ALFREDO DE SOUZA	04/05/99	05/06/14	RUA Y, 27	5874,52
12351	3	GISELE DE JESUS	15/04/99	20/03/17	RUA Z, 49	1360,66
12352	3	RAFAEL DE JESUS	10/08/98	10/08/12	RUA Z, 55	2563,44
12353	3	ROSANA DE JESUS	14/03/87	15/08/19	RUA Z, 79	4879,55
12354	4	JOSEFINA DE ALMEIDA	16/10/97	25/03/13	RUA Y, 33	4561,88
12355	4	LUCIANA DE ALMEIDA	10/02/84	28/09/11	RUA Y, 44	2345,52
12356	4	THIAGO DE ALMEIDA	10/03/98	24/10/18	RUA Y, 55	1254,22
12357	5	LARISSSA DE CAMARGO	14/02/97	04/08/15	RUA V, 22	1245,55
12358	5	ANTONIO DE CAMARGO	25/01/85	12/08/16	RUA V, 44	2451,33
12359	5	JOSE DE CAMARGO	23/10/98	20/04/17	RUA V, 88	6541,22

Figura 6 - Resultado do SELECT com SUM(coluna)
Fonte: Elaborado pela autora (2023)

1.7 Função de grupo: AVG()

A função de grupo AVG() retorna a média aritmética dos valores da coluna especificada, através de um comando SELECT. Este comando, através da função AVG (coluna), retornará a média aritmética dos valores das linhas/registros da coluna especificada, com o conteúdo diferente de nulo.

No exemplo a seguir, será recuperada a média de salários de todos os funcionários.

```
-- EXEMPLO - FUNÇÃO AVG (coluna)
SELECT AVG(VL_SALARIO_MENSAL) FROM T_SIP_FUNCIONARIO;
```

Código-fonte 9 - Exemplo de código-fonte SQL – Função AVG (coluna)
Fonte: Elaborado pela autora (2023)

Resultado da execução do comando “SELECT” com a função AVG (coluna).

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

AVG(VL_SALARIO_MENSAL)
3307,85333333333333333333333333333333333

NR_MATRICULA	CD_DEPTO	NM_FUNCIONARIO	DT_NASCIMENTO	DT_ADMISSAO	DS_ENDERECO	VL_SALARIO_MENSAL
12345	1	JOAO DA SILVA	10/05/85	15/09/12	RUA X, 49	5684,66
12346	1	MANUEL DA SILVA	05/10/98	10/11/15	RUA X, 31	3542,11
12347	1	JANDIRA DA SILVA	10/12/00	15/09/18	RUA X, 25	1875,96
12348	2	KATIA REGINA SOUZA	15/01/95	03/10/15	RUA Y, 49	3894,63
12349	2	MARIA DAS DORES SOUZA	18/08/83	23/10/17	RUA Y, 35	1542,55
12350	2	ALFREDO DE SOUZA	04/05/99	05/06/14	RUA Y, 27	5874,52
12351	3	GISELE DE JESUS	15/04/99	20/03/17	RUA Z, 49	1360,66
12352	3	RAFAEL DE JESUS	10/08/98	10/08/12	RUA Z, 55	2563,44
12353	3	ROSANA DE JESUS	14/03/87	15/08/19	RUA Z, 79	4879,55
12354	4	JOSEFINA DE ALMEIDA	16/10/97	25/03/13	RUA Y, 33	4561,88
12355	4	LUCIANA DE ALMEIDA	10/02/84	28/09/11	RUA Y, 44	2345,52
12356	4	THIAGO DE ALMEIDA	10/03/98	24/10/18	RUA Y, 55	1254,22
12357	5	LARISSSA DE CAMARGO	14/02/97	04/08/15	RUA V, 22	1245,55
12358	5	ANTONIO DE CAMARGO	25/01/85	12/08/16	RUA V, 44	2451,33
12359	5	JOSE DE CAMARGO	23/10/98	20/04/17	RUA V, 88	6541,22

Figura 7 - Resultado do SELECT com AVG (coluna)
Fonte: Elaborado pela autora (2023)

1.8 Função de grupo: MIN()

A função de grupo MIN() retorna o menor valor da coluna especificada, através de um comando SELECT. Este comando, através da função MIN(coluna), retornará o menor valor das linhas/registros da coluna especificada, com o conteúdo diferente de nulo.

No exemplo a seguir, será recuperado o menor salário, avaliando todos os funcionários de uma empresa.

```
-- EXEMPLO – FUNÇÃO MIN (coluna)
SELECT MIN(VL_SALARIO_MENSAL) FROM T_SIP_FUNCIONARIO;
```

Código-fonte 10 - Exemplo de código-fonte SQL – Função MIN(coluna)
Fonte: Elaborado pela autora (2023)

Resultado da execução do comando “SELECT” com a função MIN(coluna).

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

MIN(VL_SALARIO_MENSAL)
1245,55

NR_MATRICULA	CD_DEPTO	NM_FUNCIONARIO	DT_NASCIMENTO	DT_ADMISSAO	DS_ENDERECO	VL_SALARIO_MENSAL
12345	1	JOAO DA SILVA	10/05/85	15/09/12	RUA X, 49	5684,66
12346	1	MANUEL DA SILVA	05/10/98	10/11/15	RUA X, 31	3542,11
12347	1	JANDIRA DA SILVA	10/12/00	15/09/18	RUA X, 25	1875,96
12348	2	KATIA REGINA SOUZA	15/01/95	03/10/15	RUA Y, 49	3894,63
12349	2	MARIA DAS DORES SOUZA	18/08/83	23/10/17	RUA Y, 35	1542,55
12350	2	ALFREDO DE SOUZA	04/05/99	05/06/14	RUA Y, 27	5874,52
12351	3	GISELE DE JESUS	15/04/99	20/03/17	RUA Z, 49	1360,66
12352	3	RAFAEL DE JESUS	10/08/98	10/08/12	RUA Z, 55	2563,44
12353	3	ROSANA DE JESUS	14/03/87	15/08/19	RUA Z, 79	4879,55
12354	4	JOSEFINA DE ALMEIDA	16/10/97	25/03/13	RUA Y, 33	4561,88
12355	4	LUCIANA DE ALMEIDA	10/02/84	28/09/11	RUA Y, 44	2345,52
12356	4	THIAGO DE ALMEIDA	10/03/98	24/10/18	RUA Y, 55	1254,22
12357	5	LARISSSA DE CAMARGO	14/02/97	04/08/15	RUA V, 22	1245,55
12358	5	ANTONIO DE CAMARGO	25/01/85	12/08/16	RUA V, 44	2451,33
12359	5	JOSE DE CAMARGO	23/10/98	20/04/17	RUA V, 88	6541,22

Figura 8 - Resultado do SELECT com MIN(coluna)
Fonte: Elaborado pela autora (2023)

1.9 Função de grupo: MAX()

A função de grupo MAX() retorna o maior valor da coluna especificada, através de um comando SELECT. Este comando, através da função MAX(coluna), retornará o maior valor das linhas/registros da coluna especificada, com o conteúdo diferente de nulo.

No próximo exemplo de código-fonte, será recuperado o maior salário, avaliando todos os funcionários de uma empresa.

```
-- EXEMPLO - FUNÇÃO MAX (coluna)
SELECT MAX(VL_SALARIO_MENSAL) FROM T_SIP_FUNCIONARIO;
```

Código-fonte 11 - Exemplo de código-fonte SQL – Função MAX(coluna)
Fonte: Elaborado pela autora (2023)

Resultado da execução do comando “SELECT” com a função MAX(coluna).

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

MAX(VL_SALARIO_MENSAL)
6541,22

NR_MATRICULA	CD_DEPTO	NM_FUNCIONARIO	DT_NASCIMENTO	DT_ADMISSAO	DS_ENDERECO	VL_SALARIO_MENSAL
12345	1	JOAO DA SILVA	10/05/85	15/09/12	RUA X, 49	5684,66
12346	1	MANUEL DA SILVA	05/10/98	10/11/15	RUA X, 31	3542,11
12347	1	JANDIRA DA SILVA	10/12/00	15/09/18	RUA X, 25	1875,96
12348	2	KATIA REGINA SOUZA	15/01/95	03/10/15	RUA Y, 49	3894,63
12349	2	MARIA DAS DORES SOUZA	18/08/83	23/10/17	RUA Y, 35	1542,55
12350	2	ALFREDO DE SOUZA	04/05/99	05/06/14	RUA Y, 27	5874,52
12351	3	GISELE DE JESUS	15/04/99	20/03/17	RUA Z, 49	1360,66
12352	3	RAFAEL DE JESUS	10/08/98	10/08/12	RUA Z, 55	2563,44
12353	3	ROSANA DE JESUS	14/03/87	15/08/19	RUA Z, 79	4879,55
12354	4	JOSEFINA DE ALMEIDA	16/10/97	25/03/13	RUA Y, 33	4561,88
12355	4	LUCIANA DE ALMEIDA	10/02/84	28/09/11	RUA Y, 44	2345,52
12356	4	THIAGO DE ALMEIDA	10/03/98	24/10/18	RUA Y, 55	1254,22
12357	5	LARISSSA DE CAMARGO	14/02/97	04/08/15	RUA V, 22	1245,55
12358	5	ANTONIO DE CAMARGO	25/01/85	12/08/16	RUA V, 44	2451,33
12359	5	JOSE DE CAMARGO	23/10/98	20/04/17	RUA V, 88	6541,22

Figura 9 - Resultado do SELECT com MAX(coluna)
Fonte: Elaborado pela autora (2023)

1.10 Função de grupo: STDDEV()

A função de grupo STDDEV() retorna o desvio padrão, referente aos valores da coluna especificada, através de um comando SELECT.

O comando SELECT, através da função STDDEV (coluna), retornará o desvio padrão das linhas/registros da coluna especificada, com o conteúdo diferente de nulo.

Nota: em probabilidade e estatística, o desvio padrão é a medida mais comum de dispersão estatística. O desvio padrão é definido como a raiz quadrada da variância. Na teoria da probabilidade e na estatística, a variância de uma variável aleatória é uma medida da sua dispersão estatística, indicando quão longe em geral os seus valores se encontram do valor esperado.

No exemplo a seguir, será recuperado o desvio padrão referente aos salários, avaliando todos os funcionários de uma empresa.

```
-- EXEMPLO – FUNÇÃO STDDEV (coluna)
SELECT STDDEV(VL_SALARIO_MENSAL) FROM T_SIP_FUNCIONARIO;
```

Código-fonte 12 - Exemplo de código-fonte SQL – Função STDDEV(coluna)
Fonte: Elaborado pela autora (2023)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Resultado da execução do comando “SELECT” com a função STDDEV(coluna).

STDDEV(VL_SALARIO_MENSAL)
1827,8831253887520860507021661935250480 47

NR_MATRICULA	CD_DEPTO	NM_FUNCIONARIO	DT_NASCIMENTO	DT_ADMISSAO	DS_ENDERECO	VL_SALARIO_MENSAL
12345	1	JOAO DA SILVA	10/05/85	15/09/12	RUA X, 49	5684,66
12346	1	MANUEL DA SILVA	05/10/98	10/11/15	RUA X, 31	3542,11
12347	1	JANDIRA DA SILVA	10/12/00	15/09/18	RUA X, 25	1875,96
12348	2	KATIA REGINA SOUZA	15/01/95	03/10/15	RUA Y, 49	3894,63
12349	2	MARIA DAS DORES SOUZA	18/08/83	23/10/17	RUA Y, 35	1542,55
12350	2	ALFREDO DE SOUZA	04/05/99	05/06/14	RUA Y, 27	5874,52
12351	3	GISELE DE JESUS	15/04/99	20/03/17	RUA Z, 49	1360,66
12352	3	RAFAEL DE JESUS	10/08/98	10/08/12	RUA Z, 55	2563,44
12353	3	ROSANA DE JESUS	14/03/87	15/08/19	RUA Z, 79	4879,55
12354	4	JOSEFINA DE ALMEIDA	16/10/97	25/03/13	RUA Y, 33	4561,88
12355	4	LUCIANA DE ALMEIDA	10/02/84	28/09/11	RUA Y, 44	2345,52
12356	4	THIAGO DE ALMEIDA	10/03/98	24/10/18	RUA Y, 55	1254,22
12357	5	LARISSSA DE CAMARGO	14/02/97	04/08/15	RUA V, 22	1245,55
12358	5	ANTONIO DE CAMARGO	25/01/85	12/08/16	RUA V, 44	2451,33
12359	5	JOSE DE CAMARGO	23/10/98	20/04/17	RUA V, 88	6541,22

Figura 10 - Resultado do SELECT com STDDEV(coluna)
Fonte: Elaborado pela autora (2023)

1.11 Função de grupo: VARIANCE()

A função de grupo VARIANCE() retorna a variância referente aos valores da coluna especificada, através de um comando SELECT. Este comando, através da função VARIANCE (coluna), retornará a variância das linhas/registros da coluna especificada, com o conteúdo diferente de nulo.

Nota: na teoria da probabilidade e na estatística, a variância de uma variável aleatória é uma medida da sua dispersão estatística, indicando quão longe em geral os seus valores se encontram do valor esperado.

No exemplo a seguir, será recuperada a variância referente aos salários, avaliando todos os funcionários de uma empresa.

```
-- EXEMPLO – FUNÇÃO VARIANCE (coluna)
SELECT VARIANCE(VL_SALARIO_MENSAL) FROM T_SIP_FUNCIONARIO;
```

Código-fonte 13 - Exemplo de código-fonte SQL – Função VARIANCE(coluna)
Fonte: Elaborado pela autora (2023)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Resultado da execução do comando “SELECT” com a função VARIANCE(coluna).

VARIANCE(VL_SALARIO_MENSAL)
3341156,7200809523809523809523809523809 3

NR_MATRICULA	CD_DEPTO	NM_FUNCIONARIO	DT_NASCIMENTO	DT_ADMISSAO	DS_ENDERECO	VL_SALARIO_MENSAL
12345	1	JOAO DA SILVA	10/05/85	15/09/12	RUA X, 49	5684,66
12346	1	MANUEL DA SILVA	05/10/98	10/11/15	RUA X, 31	3542,11
12347	1	JANDIRA DA SILVA	10/12/00	15/09/18	RUA X, 25	1875,96
12348	2	KATIA REGINA SOUZA	15/01/95	03/10/15	RUA Y, 49	3894,63
12349	2	MARIA DAS DORES SOUZA	18/08/83	23/10/17	RUA Y, 35	1542,55
12350	2	ALFREDO DE SOUZA	04/05/99	05/06/14	RUA Y, 27	5874,52
12351	3	GISELE DE JESUS	15/04/99	20/03/17	RUA Z, 49	1360,66
12352	3	RAFAEL DE JESUS	10/08/98	10/08/12	RUA Z, 55	2563,44
12353	3	ROSANA DE JESUS	14/03/87	15/08/19	RUA Z, 79	4879,55
12354	4	JOSEFINA DE ALMEIDA	16/10/97	25/03/13	RUA Y, 33	4561,88
12355	4	LUCIANA DE ALMEIDA	10/02/84	28/09/11	RUA Y, 44	2345,52
12356	4	THIAGO DE ALMEIDA	10/03/98	24/10/18	RUA Y, 55	1254,22
12357	5	LARISSSA DE CAMARGO	14/02/97	04/08/15	RUA V, 22	1245,55
12358	5	ANTONIO DE CAMARGO	25/01/85	12/08/16	RUA V, 44	2451,33
12359	5	JOSE DE CAMARGO	23/10/98	20/04/17	RUA V, 88	6541,22

Figura 11 - Resultado do SELECT com VARIANCE(coluna)
Fonte: Elaborado pela autora (2023)

1.12 Visualizando a aplicação de um agrupamento

Podemos agrupar um conjunto de linhas/registros a partir de colunas e aplicar funções de grupo nesses conjuntos de dados.

Um agrupamento possibilita a apresentação de um resultado para cada agrupamento especificado.

Pontos de atenção:

- Para limitar o resultado de linhas que será envolvido no agrupamento, deve-se utilizar a cláusula WHERE e, em seguida, a cláusula GROUP BY.
- Todas as colunas individuais envolvidas na consulta, isto é, que não estão participando de funções de grupo, devem ser incluídas na cláusula GROUP BY.
- A função de grupo não deve estar no GROUP BY.
- A coluna GROUP BY não precisa estar na cláusula SELECT.
- Pode-se usar a função de grupo na cláusula ORDER BY.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

A imagem a seguir ilustra o agrupamento e a contagem de linhas/registros feitos através da função e o retorno, ou seja, a contagem retornada através da função de grupo.

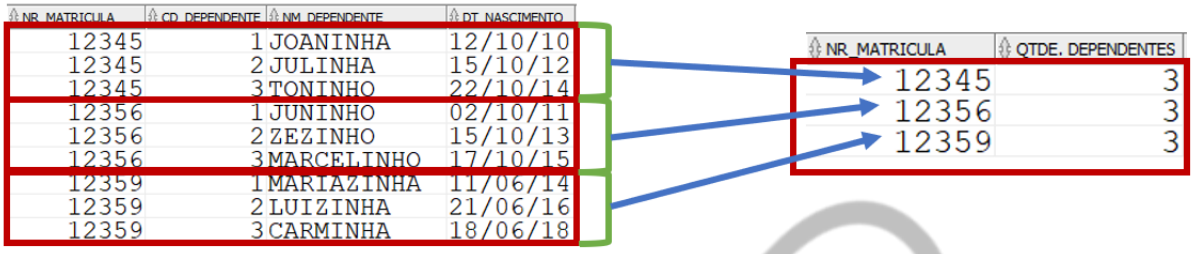


Figura 12 - Representação de um agrupamento
Fonte: Elaborado pela autora (2023)

1.13 Criando um agrupamento de dados

A cláusula GROUP BY permite que criemos um grupo de dados. Podemos aplicar a função de grupo para extrair informações relacionadas ao agrupamento criado.

O comando SELECT, através da cláusula GROUP BY, retornará o grupo de dados e aplicada da função de grupo, caso esteja definida no comando SELECT.

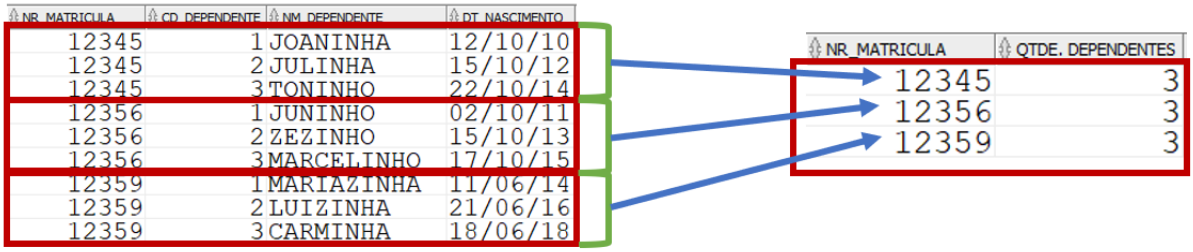
No exemplo a seguir, serão recuperados a quantidade de filhos por funcionário. O agrupamento foi feito a partir do funcionário e aplicada a função de grupo, para que fossem contados a quantidade de dependentes de cada funcionário.

```
-- EXEMPLO – GROUP BY
  SELECT NR_MATRICULA,
         COUNT(CD_DEPENDENTE) "QTDE. DEPENDENTES"
    FROM T_SIP_DEPENDENTE
GROUP BY NR_MATRICULA;
```

Código-fonte 14 - Exemplo de código-fonte SQL – GROUP BY
Fonte: Elaborado pela autora (2023)

Resultado da execução do comando “SELECT” com o GROUP BY.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com



NR_MATRICULA	CD_DEPENDENTE	NM_DEPENDENTE	DT_NASCIMENTO
12345	1	JOANINHA	12/10/10
12345	2	JULINHA	15/10/12
12345	3	TONINHO	22/10/14
12356	1	JUNINHO	02/10/11
12356	2	ZEZINHO	15/10/13
12356	3	MARCELINHO	17/10/15
12359	1	MARIAZINHA	11/06/14
12359	2	LUIZINHA	21/06/16
12359	3	CARMINHA	18/06/18

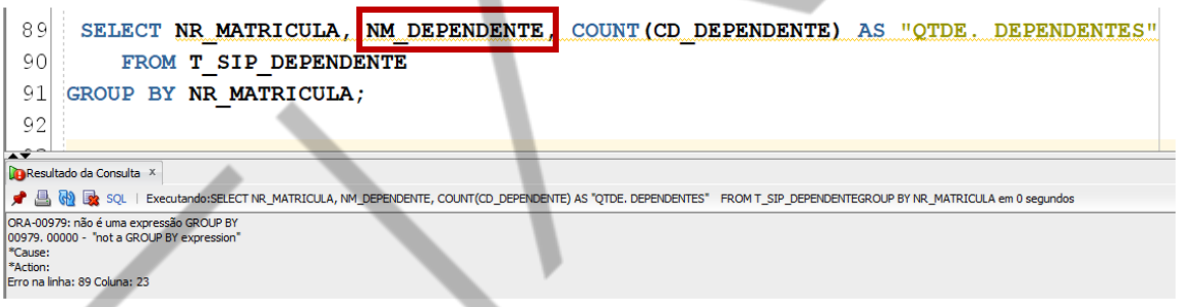
NR_MATRICULA	QTDE. DEPENDENTES
12345	3
12356	3
12359	3

Figura 13 - Resultado do SELECT com GROUP BY
Fonte: Elaborado pela autora (2023)

Podemos nos deparar com erros, durante a execução de um comando SELECT com GROUP BY e funções de grupo.

No próximo exemplo, a coluna “NM_DEPENDENTE” não foi inserida no GROUP BY, o que causou o erro.

Qualquer coluna ou expressão na lista “SELECT” que não seja uma função de grupo, deve estar presente na cláusula “GROUP BY”.



```
89  SELECT NR_MATRICULA, NM_DEPENDENTE, COUNT(CD_DEPENDENTE) AS "QTDE. DEPENDENTES"
90     FROM T_SIP_DEPENDENTE
91 GROUP BY NR_MATRICULA;
92
```

Resultado da Consulta

SQL | Executando:SELECT NR_MATRICULA, NM_DEPENDENTE, COUNT(CD_DEPENDENTE) AS "QTDE. DEPENDENTES" FROM T_SIP_DEPENDENTEGROUP BY NR_MATRICULA em 0 segundos

```
ORA-00979: não é uma expressão GROUP BY
00979. 00000 - "not a GROUP BY expression"
*Cause:
*Action:
Erro na linha: 89 Coluna: 23
```

Figura 14 - Erros durante a execução de agrupamentos
Fonte: Elaborado pela autora (2023)

1.14 Criando um agrupamento de dados apenas com funções de grupo

A cláusula GROUP BY permite que criemos um grupo de dados. Podemos aplicar a função de grupo para extrair informações relacionadas ao agrupamento criado.

O comando SELECT, através da cláusula GROUP BY, retornará o grupo de dados e aplicada da função de grupo, caso esteja definida no comando SELECT.

No exemplo a seguir, serão recuperados: o total de funcionários, somatória dos salários e a média salarial por departamento. O agrupamento foi feito a partir do

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

departamento e aplicadas as funções de grupo para que fossem calculados: total de funcionários, somatória e a média salarial.

Como temos apenas a função de grupo no comando SELECT, teremos apenas a coluna responsável pelo agrupamento na cláusula GROUP BY.

```
-- EXEMPLO – GROUP BY COM FUNÇÃO DE GRUPO
  SELECT CD_DEPTO,
         COUNT(NR_MATRICULA) "QTDE. FUNCIONARIOS NO DEPTO",
         SUM(VL_SALARIO_MENSAL) "TOTAL SALARIO POR DEPTO",
         ROUND(AVG(VL_SALARIO_MENSAL),2) "MEDIA SALARIAL POR
DEPTO"
    FROM T_SIP_FUNCIONARIO
GROUP BY CD_DEPTO
ORDER BY CD_DEPTO;
```

Código-fonte 15 - Exemplo de código-fonte SQL – GROUP BY e função de grupo
Fonte: Elaborado pela autora (2023)

Resultado da execução do comando “SELECT” com o GROUP BY.

CD_DEPTO	QTDE. FUNCIONARIOS NO DEPTO	TOTAL SALARIO POR DEPTO	MEDIA SALARIAL POR DEPTO
1	3	11102,73	3700,91
2	3	11311,7	3770,57
3	3	8803,65	2934,55
4	3	8161,62	2720,54
5	3	10238,1	3412,7

NR_MATRICULA	CD_DEPTO	NM_FUNCIONARIO	DT_NASCIMENTO	DT_ADMISSAO	DS_ENDERECO	VL_SALARIO_MENSAL
12345	1	JOAO DA SILVA	10/05/85	15/09/12	RUA X, 49	5684,66
12346	1	MANUEL DA SILVA	05/10/98	10/11/15	RUA X, 31	3542,11
12347	1	JANDIRA DA SILVA	10/12/00	15/09/18	RUA X, 25	1875,96
12348	2	KATIA REGINA SOUZA	15/01/95	03/10/15	RUA Y, 49	3894,63
12349	2	MARIA DAS DORES SOUZA	18/08/83	23/10/17	RUA Y, 35	1542,55
12350	2	ALFREDO DE SOUZA	04/05/99	05/06/14	RUA Y, 27	5874,52
12351	3	GISELE DE JESUS	15/04/99	20/03/17	RUA Z, 49	1360,66
12352	3	RAFAEL DE JESUS	10/08/98	10/08/12	RUA Z, 55	2563,44
12353	3	ROSANA DE JESUS	14/03/87	15/08/19	RUA Z, 79	4879,55
12354	4	JOSEFINA DE ALMEIDA	16/10/97	25/03/13	RUA Y, 33	4561,88
12355	4	LUCIANA DE ALMEIDA	10/02/84	28/09/11	RUA Y, 44	2345,52
12356	4	THIAGO DE ALMEIDA	10/03/98	24/10/18	RUA Y, 55	1254,22
12357	5	LARISSSA DE CAMARGO	14/02/97	04/08/15	RUA V, 22	1245,55
12358	5	ANTONIO DE CAMARGO	25/01/85	12/08/16	RUA V, 44	2451,33
12359	5	JOSE DE CAMARGO	23/10/98	20/04/17	RUA V, 88	6541,22

Figura 15 - Resultado do SELECT com GROUP BY e função de grupo
Fonte: Elaborado pela autora (2023)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1.15 Criando um agrupamento de dados com várias tabelas

A cláusula GROUP BY permite que criemos um grupo de dados. Podemos aplicar a função de grupo para extrair informações relacionadas ao agrupamento criado, e ainda consultar duas ou mais tabelas, de acordo com a necessidade de negócio apresentada.

O comando SELECT, por meio das junções, combinará as tabelas, e pela cláusula GROUP BY, retornará o grupo de dados e aplicada da função de grupo, caso esteja definida no comando SELECT.

No exemplo a seguir, a combinação das tabelas “FUNCIONARIO” e “DEPENDENTE” permitirá a recuperação do nome do funcionário e não apenas o número de matrícula. A partir do agrupamento serão recuperadas a quantidade de dependentes por funcionário.

Como temos, além da função de grupo, as colunas “nm_funcionario” e “nr_matricula”, é necessário que estas colunas estejam na cláusula GROUP BY, caso contrário, ocorrerá um erro.

```
-- EXEMPLO - GROUP BY COM VÁRIAS TABELAS
      SELECT F.NM_FUNCIONARIO        "FUNCIONARIO" ,
             D.NR_MATRICULA          "MATRICULA"   ,
             COUNT(D.NR_MATRICULA)   "QTDE. DEPENDENTES"
        FROM T_SIP_FUNCIONARIO F INNER JOIN T_SIP_DEPENDENTE D
             ON (F.NR_MATRICULA = D.NR_MATRICULA)
    GROUP BY F.NM_FUNCIONARIO, D.NR_MATRICULA;
```

Código-fonte 16 - Exemplo de código-fonte SQL – GROUP BY e várias tabelas
Fonte: Elaborado pela autora (2023)

Resultado da execução do comando “SELECT” com o GROUP BY e várias tabelas.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

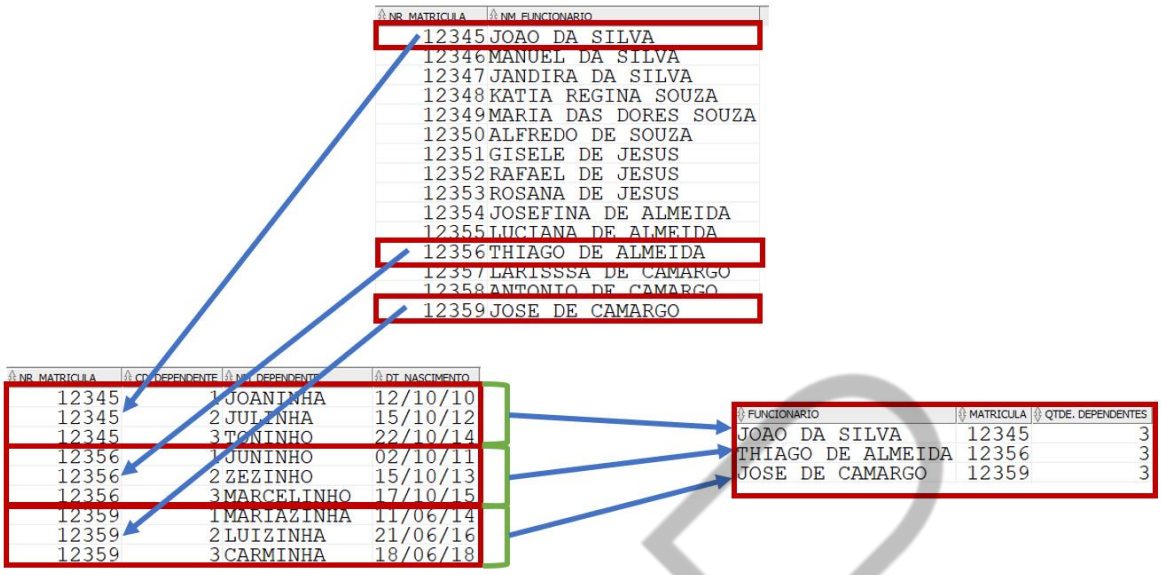


Figura 16 - Resultado do SELECT com GROUP BY e várias tabelas
Fonte: Elaborado pela autora (2023)

1.16 Cláusula HAVING

Quando utilizamos funções de grupo e agrupamentos, temos a necessidade de estabelecer condições para recuperar o resultado desejado.

Podemos utilizar a cláusula WHERE para selecionar as linhas/registros que participarão do agrupamento. A cláusula WHERE não pode restringir grupos.

Podemos utilizar a cláusula HAVING para aplicar uma condição de seleção sobre as linhas agrupadas. A cláusula **HAVING** é utilizada para realizar restrições ao agrupamento com a cláusula **GROUP BY**. Nessa cláusula, podemos usar funções de agrupamento: AVG, COUNT, MIN, MAX, SUM, por exemplo.

No próximo exemplo de código-fonte, serão recuperados os seguintes dados: total de funcionários, somatória e a média salarial por departamento. O agrupamento foi feito a partir do departamento e aplicadas as funções de grupo para que fossem calculados total de funcionários, somatória e a média salarial. Como temos a cláusula HAVING, serão filtrados os agrupamentos formados, recuperando apenas os agrupamentos onde a somatória de salários for superior a R$ 10.000,00.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
-- EXEMPLO – GROUP BY COM HAVING
  SELECT CD_DEPTO,
         COUNT(NR_MATRICULA) "QTDE. FUNCIONARIO NO DEPTO",
         SUM(VL_SALARIO_MENSAL) "TOTAL SALARIO POR DEPTO",
         ROUND(AVG(VL_SALARIO_MENSAL),2) "MEDIA SALARIAL POR
DEPTO"
    FROM T_SIP_FUNCIONARIO
GROUP BY CD_DEPTO
HAVING   SUM(VL_SALARIO_MENSAL) > 10000;
```

Código-fonte 17 - Exemplo de código-fonte SQL – GROUP BY com HAVING
Fonte: Elaborado pela autora (2023)

Resultado da execução do comando “SELECT” com o GROUP BY e a cláusula HAVING.

CD_DEPTO	QTDE. FUNCIONARIO NO DEPTO	TOTAL SALARIO POR DEPTO	MEDIA SALARIAL POR DEPTO
2	3	11311,7	3770,57
5	3	10238,1	3412,7
1	3	11102,73	3700,91

NR_MATRICULA	CD_DEPTO	NM_FUNCIONARIO	DT_NASCIMENTO	DT_ADMISSAO	DS_ENDERECO	VL_SALARIO_MENSAL
12345	1	JOAO DA SILVA	10/05/85	15/09/12	RUA X, 49	5684,66
12346	1	MANUEL DA SILVA	05/10/98	10/11/15	RUA X, 31	3542,11
12347	1	JANDIRA DA SILVA	10/12/00	15/09/18	RUA X, 25	1875,96
12348	2	KATIA REGINA SOUZA	15/01/95	03/10/15	RUA Y, 49	3894,63
12349	2	MARIA DAS DORES SOUZA	18/08/83	23/10/17	RUA Y, 35	1542,55
12350	2	ALFREDO DE SOUZA	04/05/99	05/06/14	RUA Y, 27	5874,52
12351	3	GISELE DE JESUS	15/04/99	20/03/17	RUA Z, 49	1360,66
12352	3	RAFAEL DE JESUS	10/08/98	10/08/12	RUA Z, 55	2563,44
12353	3	ROSANA DE JESUS	14/03/87	15/08/19	RUA Z, 79	4879,55
12354	4	JOSEFINA DE ALMEIDA	16/10/97	25/03/13	RUA Y, 33	4561,88
12355	4	LUCIANA DE ALMEIDA	10/02/84	28/09/11	RUA Y, 44	2345,52
12356	4	THIAGO DE ALMEIDA	10/03/98	24/10/18	RUA Y, 55	1254,22
12357	5	LARISSSA DE CAMARGO	14/02/97	04/08/15	RUA V, 22	1245,55
12358	5	ANTONIO DE CAMARGO	25/01/85	12/08/16	RUA V, 44	2451,33
12359	5	JOSE DE CAMARGO	23/10/98	20/04/17	RUA V, 88	6541,22

Figura 17 - Resultado do SELECT com o GROUP BY com HAVING
Fonte: Elaborado pela autora (2023)

Podemos nos deparar com erros durante a execução de um comando SELECT com GROUP BY e funções de grupo.

No próximo exemplo, foi inserida a validação de uma função de grupo, na cláusula WHERE, o que acarretou o erro. Os filtros de agrupamento só podem ser

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

feitos utilizando a cláusula HAVING. Portanto, não é possível usar a cláusula “WHERE” para restringir grupos.

```
132 -- ERRO VALIDAÇÃO FUNÇÃO DE GRUPO NO WHERE
133   SELECT CD_DEPTO,
134          COUNT(NR_MATRICULA) "QTDE. FUNCIONARIO NO DEPTO",
135          SUM(VL_SALARIO_MENSAL) "TOTAL SALARIO POR DEPTO",
136          ROUND(AVG(VL_SALARIO_MENSAL),2) "MEDIA SALARIAL POR DEPTO"
137     FROM T_SIP_FUNCIONARIO
138 GROUP BY CD_DEPTO
139    WHERE SUM(VL_SALARIO_MENSAL) > 10000;
140
```

```
Resultado da Consulta
SQL | Executando:SELECT CD_DEPTO,    COUNT(NR_MATRICULA) "QTDE. FUNCIONARIO NO DEPTO",    SUM(VL_SALARIO_MENSAL) "TOTAL SALARIO POR DEPTO",
ORA-00933: comando SQL não encerrado adequadamente
00933. 00000 - "SQL command not properly ended"
*Cause:
*Action:
Erro na linha: 139 Coluna: 4
```

Figura 18 - Erros durante a execução de agrupamentos com WHERE
Fonte: Elaborado pela autora (2023)

1.17 Desvio Condicional (Instrução CASE)

Podemos criar desvios condicionais, através da estrutura “CASE”, utilizando PL/SQL (Procedural Language/Structured Query Language), linguagem de programação para o SGBDR Oracle, que é uma extensão da linguagem SQL.

A sintaxe básica da instrução “CASE” é apresentada a seguir:

```
1  CASE
2      WHEN <CONDIÇÃO 1> THEN <VALOR 1>
3      WHEN <CONDIÇÃO 2> THEN <VALOR 2>
4      WHEN <CONDIÇÃO 3> THEN <VALOR 3>
5      .
6      .
7      .
8      ELSE <VALOR 4>
9  END
```

Código-fonte 18 - Exemplo de instrução CASE
Fonte: Elaborado pela autora (2023)

No próximo exemplo, serão recuperados o status dos projetos “EM ANDAMENTO” ou “FINALIZADO”, baseado na validação da “DT_TERMINO”, ou seja,

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

caso a “DT_TERMINO IS NULL” (data término nula) indica que o projeto está em andamento, caso contrário, data término preenchida, indica que o projeto está finalizado.

Utilizaremos a instrução “CASE” para realizar a validação da “DT_TERMINO” e apresentação dos respectivos status.

```
-- EXEMPLO – INSTRUÇÃO CASE
SELECT
        NM_PROJETO,
        DT_TERMINO,
CASE
  WHEN DT_TERMINO IS NULL THEN 'EM ANDAMENTO'
  ELSE 'FINALIZADO'
END STATUS_PROJETO
  FROM T_SIP_PROJETO
ORDER BY NM_PROJETO;
```

Código-fonte 19 - Exemplo de código-fonte SQL – Instrução CASE
Fonte: Elaborado pela autora (2023)

Resultado da execução do comando “SELECT” com a instrução “CASE”.

NM_PROJETO	DT_TERMINO	STATUS_PROJETO
PROJETO 1	(null)	EM ANDAMENTO
PROJETO 2	15/04/15	FINALIZADO
PROJETO 3	(null)	EM ANDAMENTO
PROJETO 4	(null)	EM ANDAMENTO
PROJETO 5	15/09/18	FINALIZADO
PROJETO 6	(null)	EM ANDAMENTO

Figura 19 - Resultado do SELECT com a instrução CASE
Fonte: Elaborado pela autora (2023)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1.18 Hands On – Consulta ou Recuperação de dados utilizando Grupos e Agrupamentos

1.18.1 Estudo de Caso: SIP – SISTEMA DE CONTROLE DE IMPLANTACAO DE PROJETOS

1.18.1.1 Recuperando dados utilizando Grupos e Agrupamentos

No tópico “Exemplos de aplicação do comando SELECT utilizando Grupos e Agrupamentos”, utilizamos o “SCRIPT DDL” e “SCRIPT DML”, para criar a estrutura do estudo de caso SIP e popular as tabelas deste projeto.

Iremos consultar os dados das tabelas deste estudo de caso, para fixarmos os comandos trabalhados nas seções anteriores.

Como sugestão, podemos utilizar a ferramenta SQL Developer da Oracle.

O nome do SCRIPT (arquivo com comandos SQL) deve possuir um nome que ajude na sua identificação, por exemplo: “SCRIPT_DRS-GRUPOS_PROJETO_SIP.SQL”, com a extensão SQL.

Consulta 1

Escreva e execute a instrução SQL para:

Recuperar a implantação com maior tempo de duração.

Mostrar: o tempo de duração em dias e concatenar com a palavra ' DIAS'.

Cálculo para obter o tempo de duração: DT_SAIDA - DT_ENTRADA

Utilize a função de grupo MAX().

```
-- CONSULTA 1
SELECT MAX(DT_SAIDA - DT_ENTRADA) || ' DIAS'
  FROM T_SIP_IMPLANTACAO;
```

Código-fonte 20 - Exemplo de código-fonte SQL – comando SELECT com função de grupo – Consulta 1
Fonte: Elaborado pela autora (2023)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Resultado da execução a consulta, através do comando “SELECT”:

MAX(DT_SAIDA-DT_ENTRADA)||'DIAS'
1448 DIAS

Figura 20 - Resultado da Consulta 1
Fonte: Elaborado pela autora (2023)

Consulta 2

Escreva e execute a instrução SQL para:

Recuperar a média salarial por departamento.

Mostrar: código e nome do departamento e a média salarial por departamento, utilize o apelido "MÉDIA SALARIAL POR DEPTO".

Apresente o resultado ordenado pela média salarial obtida.

Utilize o conceito de junção de tabelas.

Utilize a função de grupo AVG().

Crie agrupamentos por: código e nome do departamento.

```
-- CONSULTA 2
  SELECT F.CD_DEPTO, D.NM_DEPTO,
         ROUND(AVG(VL_SALARIO_MENSAL),2)  "MÉDIA  SALARIAL  POR
DEPTO"
    FROM T_SIP_DEPARTAMENTO D INNER JOIN T_SIP_FUNCIONARIO F
         ON (D.CD_DEPTO = F.CD_DEPTO)
GROUP BY F.CD_DEPTO, D.NM_DEPTO
ORDER BY "MÉDIA SALARIAL POR DEPTO" DESC;
```

Código-fonte 21 - Exemplo de código-fonte SQL – comando SELECT com função de grupo e agrupamento – Consulta 2
Fonte: Elaborado pela autora (2023)

Resultado da execução a consulta, através do comando “SELECT”:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

CD_DEPTO	NM_DEPTO	MÉDIA SALARIAL POR DEPTO
2	TECNOLOGIA DA INFORMAÇÃO	3770,57
1	FINANCEIRO	3700,91
5	RECURSOS HUMANOS	3412,7
3	CONTAS A PAGAR	2934,55
4	FATURAMENTO	2720,54

Figura 21 - Resultado da Consulta 2
Fonte: Elaborado pela autora (2023)

Consulta 3

Escreva e execute a instrução SQL para:

Recuperar a quantidade de implantações por projeto

Mostrar: código e nome do projeto e a quantidade de implantações, utilize o apelido "QTDE. IMPLANTAÇÕES".

Utilize o conceito de junção de tabelas.

Utilize a função de grupo COUNT().

Crie agrupamentos por: código e nome do projeto.

```
-- CONSULTA 3
  SELECT I.CD_PROJETO, P.NM_PROJETO,
         COUNT (I.CD_IMPLANTACAO) "QTDE.  IMPLANTAÇÕES"
    FROM T_SIP_PROJETO P INNER JOIN T_SIP_IMPLANTACAO I
         ON (P.CD_PROJETO = I.CD_PROJETO)
GROUP BY I.CD_PROJETO, P.NM_PROJETO;
```

Código-fonte 22 - Exemplo de código-fonte SQL – comando SELECT com função de grupo e agrupamento – Consulta 3
Fonte: Elaborado pela autora (2023)

Resultado da execução a consulta, através do comando “SELECT”:

CD_PROJETO	NM_PROJETO	QTDE. IMPLANTAÇÕES
10001	PROJETO 1	3
10002	PROJETO 2	3
10003	PROJETO 3	3
10004	PROJETO 4	3
10005	PROJETO 5	3

Figura 22 - Resultado da Consulta 3
Fonte: Elaborado pela autora (2023)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Consulta 4

Escreva e execute a instrução SQL para:

Recuperar a quantidade de implantações por projeto em andamento.

Mostrar: código e nome do projeto e a quantidade de implantações em andamento, utilize o apelido "QTDE. IMPLANTAÇÕES EM ANDAMENTO".

Utilize o conceito de junção de tabelas.

Utilize a função de grupo COUNT().

Crie agrupamentos por: código e nome do projeto.

```
-- CONSULTA 4
  SELECT I.CD_PROJETO, P.NM_PROJETO,
         COUNT (I.CD_IMPLANTACAO) "QTDE. IMPLANTAÇÕES EM
ANDAMENTO"
    FROM T_SIP_PROJETO P INNER JOIN T_SIP_IMPLANTACAO I
         ON (P.CD_PROJETO = I.CD_PROJETO)
   WHERE I.DT_SAIDA IS NULL
GROUP BY I.CD_PROJETO, P.NM_PROJETO;
```

Código-fonte 23 - Exemplo de código-fonte SQL – comando SELECT com função de grupo e agrupamento – Consulta 4
Fonte: Elaborado pela autora (2023)

Resultado da execução a consulta, através do comando “SELECT”:

CD_PROJETO	NM_PROJETO	QTDE. IMPLANTAÇÕES EM ANDAMENTO
10001	PROJETO 1	1
10003	PROJETO 3	3
10004	PROJETO 4	2

Figura 23 - Resultado da Consulta 4
Fonte: Elaborado pela autora (2023)

Consulta 5

Escreva e execute a instrução SQL para:

Recuperar a quantidade de implantações por projeto finalizadas.

Mostrar: código e nome do projeto e a quantidade de implantações finalizadas, utilize o apelido "QTDE. IMPLANTAÇÕES EM FINALIZADAS".

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Utilize o conceito de junção de tabelas.

Utilize a função de grupo COUNT().

Crie agrupamentos por: código e nome do projeto.

```
-- CONSULTA 5
  SELECT I.CD_PROJETO, P.NM_PROJETO,
         COUNT    (I.CD_IMPLANTACAO)    "QTDE.    IMPLANTAÇÕES
FINALIZADAS"
    FROM T_SIP_PROJETO P INNER JOIN T_SIP_IMPLANTACAO I
         ON (P.CD_PROJETO = I.CD_PROJETO)
   WHERE I.DT_SAIDA IS NOT NULL
GROUP BY I.CD_PROJETO, P.NM_PROJETO;
```

Código-fonte 24 - Exemplo de código-fonte SQL – comando SELECT com função de grupo e agrupamento – Consulta 5
Fonte: Elaborado pela autora (2023)

Resultado da execução a consulta, através do comando “SELECT”:

CD_PROJETO	NM_PROJETO	QTDE. IMPLANTAÇÕES FINALIZADAS
10001	PROJETO 1	2
10002	PROJETO 2	3
10004	PROJETO 4	1
10005	PROJETO 5	3

Figura 24 - Resultado da Consulta 5
Fonte: Elaborado pela autora (2023)

Consulta 6

Escreva e execute a instrução SQL para:

Recuperar a quantidade de implantações realizadas por funcionários.

Mostrar: código e nome do funcionário e a quantidade de implantações, utilize o apelido "QTDE. IMPLANTAÇÕES".

Utilize o conceito de junção de tabelas.

Utilize a função de grupo COUNT().

Crie agrupamentos por: código e nome do funcionário.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
-- CONSULTA 6
  SELECT I.NR_MATRICULA, F.NM_FUNCIONARIO,
         COUNT (I.CD_IMPLANTACAO) "QTDE.  IMPLANTAÇÕES"
    FROM T_SIP_FUNCIONARIO F INNER JOIN T_SIP_IMPLANTACAO I
         ON (F.NR_MATRICULA = I.NR_MATRICULA)
GROUP BY I.NR_MATRICULA, F.NM_FUNCIONARIO;
```

Código-fonte 25 - Exemplo de código-fonte SQL – comando SELECT com função de grupo e agrupamento – Consulta 6
Fonte: Elaborado pela autora (2023)

Resultado da execução a consulta, através do comando “SELECT”:

NR_MATRICULA	NM_FUNCIONARIO	QTDE. IMPLANTAÇÕES
12345	JOAO DA SILVA	4
12346	MANUEL DA SILVA	1
12348	KATIA REGINA SOUZA	1
12350	ALFREDO DE SOUZA	1
12352	RAFAEL DE JESUS	1
12354	JOSEFINA DE ALMEIDA	3
12355	LUCIANA DE ALMEIDA	2
12357	LARISSSA DE CAMARGO	1
12358	ANTONIO DE CAMARGO	1

Figura 25 - Resultado da Consulta 6
Fonte: Elaborado pela autora (2023)

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

ELMASRI, R.; NAVATHE, S.B. **Sistemas de Banco de Dados**: Fundamentos e Aplicações. 6. ed. São Paulo: Pearson Addison Wesley, 2011.

HEUSER, C.A. **Projeto de Banco de Dados**. Série Livros Didáticos. v. 4. 6. ed. Rio Grande do Sul: Bookman, 2009.

MACHADO, Felipe Nery R. **Banco de Dados - Projeto e Implementação**. 2. ed. São Paulo: Érica, 2008.

PRICE, J. **Oracle Database 11g SQL**. São Paulo: Bookman, 2009.

PUGA, S.; FRANÇA, E.; GOYA, M. **Banco de Dados**: Implementação em SQL, PL/SQL e Oracle 11g. São Paulo: Pearson Education do Brasil, 2013.

SILBERSCHATZ, A; KORTH, H. F.; SUDARSHAN, S. **Sistema de Banco de Dados**. 5. ed. Rio de Janeiro: Campus, 2006.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

GLOSSÁRIO

DRS	Data Retrieve Language (Linguagem de recuperação de dados).
DQL	Data Query Language (Linguagem para consulta de dados).
SELECT	Comando da linguagem SQL, responsável por permitir a consulta ou recuperação de registros ou linhas nas tabelas.
GROUP BY	Permite criar grupos de dados.
HAVING	Similar à cláusula WHERE, aplicado somente a agrupamentos. Permite filtrar os grupos a serem recuperados.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com